Teoria współbieżności

Sprawozdanie z laboratorium 4

Adrian Mikoda

9.11.2026 r.

Spis treści

1. Dane techniczne 2
2. Treść zadania 2
3. Realizacja ćwiczenia 2
4. Wyniki 7
5. Wnioski 8
Bibliografia 9

1. Dane techniczne

- System operacyjny: `Windows 11 Home`
- Procesor: `AMD Ryzen 5 7530U (2,0 - 4,5 GHz)`
- Pamięć RAM: `16 GB LPDDR4 (2133 MHz)`
- SDK: `Microsoft OpenJDK 25.0.4`
- Wizualizacja wyników: `Python 3.13.5`
- Użyte biblioteki: `matplotlib 3.11.2, numpy 2.5.3, pandas 3.1.0`

2. Treść zadania

Producenci i konsumenci z losową ilością pobieranych i wstawianych porcji

- Bufor o rozmiarze $2M$
- Jest m producentów i n konsumentów
- Producent wstawia do bufora losową liczbę elementów (nie więcej niż M)
- Konsument pobiera losową liczbę elementów (nie więcej niż M)
- Zaimplementować przy pomocy monitorów Javy oraz mechanizmów Java Concurrency Utilities
- Przeprowadzić porównanie wydajności (np. czas wykonywania) vs. różne parametry, zrobić wykresy i je skomentować

3. Realizacja ćwiczenia

Do implementacji rozwiązania wykorzystano gotową klasę `ReentrantLock` oraz interfejs `Condition` z pakietu `java.util.concurrent` (fragment kodu 1).

Klasy `Buffer` (fragment kodu 2), `Producer` (fragment kodu 3), `Consumer` (fragment kodu 4) oraz `PCUtil` (fragment kodu 5) zaimplementowano w pliku `PCUtil.java`.

```java
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.Random;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

import static java.util.Arrays.stream;
```

Fragment kodu 1: Plik `PCUtil.java` - wykorzystane pakiety

```java
class Buffer {
    protected final int[] buffer;
    protected final int bufferSize;
    protected int size = 0, firstIndex = 0;
    private final Lock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    private final Random random = new Random();

    public Buffer(int bufferSize) {
        this.bufferSize = bufferSize;
        this.buffer = new int[bufferSize];
    }

    public void put(int n) throws InterruptedException {
        lock.lock();
        try {
            while (size + n > bufferSize) {
                notFull.await();
            }
            for (int i = 0; i < n; ++i) {
                buffer[(firstIndex + size++) % bufferSize] = random.nextInt();
            }
            notEmpty.signalAll();
        }
        finally {
            lock.unlock();
        }
    }

    public int[] get(int n) throws InterruptedException {
        lock.lock();
        int[] result = new int[n];
        try {
            while(size < n) {
                notEmpty.await();
            }
            for (int i = 0; i < n; ++i) {
                result[i] = buffer[firstIndex];
                firstIndex = (firstIndex + 1) % bufferSize;
                size--;
            }
            notFull.signalAll();
        }
        finally {
            lock.unlock();
        }

        return result;
    }
}
```

Fragment kodu 2: Plik `PCUtil.java` - klasa `Buffer`

```java
class Producer implements Runnable {
    private final int id, M;
    private final Buffer buffer;
    private final Random random = new Random();
    private final long[][] stats;

    public Producer(int id, int M, Buffer buffer, long[][] stats) {
```

```
        this.id = id;
        this.M = M;
        this.buffer = buffer;
        this.stats = stats;
    }

    public void run() {
        try {
            while (true) {
                int n = random.nextInt(M) + 1;
                long start = System.nanoTime();
                buffer.put(n);
                long end = System.nanoTime();
                stats[0][id] += end - start;
                stats[1][id]++;
                Thread.sleep(random.nextInt(150));
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt();}
    }
}
```

Fragment kodu 3: Plik `PCUtil.java` - klasa `Producer`

```
class Consumer implements Runnable {
    private final int id, M;
    private final Buffer buffer;
    private final Random random = new Random();
    private final long[][] stats;

    public Consumer(int id, int M, Buffer buffer, long[][] stats) {
        this.id = id;
        this.M = M;
        this.buffer = buffer;
        this.stats = stats;
    }

    public void run() {
        try {
            while (true) {
                int n = random.nextInt(M) + 1;
                long start = System.nanoTime();
                int[] result = buffer.get(n);
                long end = System.nanoTime();
                stats[0][id] += end - start;
                stats[1][id]++;
                stats[2][id] += n;
                Thread.sleep(random.nextInt(150));
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
```

Fragment kodu 4: Plik `PCUtil.java` - klasa `Consumer`

```java
package lab4;

public class PCUtil {
    private static String test(int m, int n, int M) throws InterruptedException {
        final int MEASURE_MS = 5000;
        final int REPEATS = 5;

        double averageProducerWaitTime = 0;
        double averageConsumerWaitTime = 0;
        long throughput = 0;
        for (int i = 0; i < REPEATS; i++) {
            Buffer buffer = new Buffer(2*M);
            long[][] producerStats = new long[2][m];
            long[][] consumerStats = new long[3][n];
            ExecutorService executorService = Executors.newFixedThreadPool(m + n);
            for (int j = 0; j < m; j++) {
                executorService.submit(new Producer(j, M, buffer, producerStats));
            }
            for (int j = 0 ; j < n; j++) {
                executorService.submit(new Consumer(j, M, buffer, consumerStats));
            }
            executorService.shutdown();
            Thread.sleep(MEASURE_MS);
            executorService.shutdownNow();
            averageProducerWaitTime += (double) stream(producerStats[0]).sum() /
(stream(producerStats[1]).sum() * 1000000000);
            averageConsumerWaitTime += (double) stream(consumerStats[0]).sum() /
(stream(consumerStats[1]).sum() * 1000000000);
            throughput += stream(consumerStats[2]).sum();
        }
        averageProducerWaitTime /= REPEATS;
        averageConsumerWaitTime /= REPEATS;

        return "%d,%d,%d,%f,%f,
%d".formatted(m,n,M,averageProducerWaitTime,averageConsumerWaitTime,throughput);
    }

    public static void main(String[] args) {
        String path = "../results.csv";
        try {
            Files.deleteIfExists(Paths.get(path));
        } catch (Exception e) {
            e.printStackTrace();
        }

        int[] Ms = {100, 500, 1000, 5000};
        int[] workerSizes = {1, 5, 10, 50, 100, 500};
        String content =
"m,n,M,avg_producer_wait_time,avg_consumer_wait_time,throughput" + System.lineSeparator();
        try {
            Files.write(Paths.get(path), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
        } catch (Exception e) {e.printStackTrace();}
        try {
            for (int M : Ms) {
                for (int m : workerSizes) {
                    for (int n : workerSizes) {
                        content = test(m, n, M) + System.lineSeparator();
                        try {
                            Files.write(Paths.get(path), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

```
56                          } catch (Exception e) {e.printStackTrace();}
57                      }
58                  }
59              }
60          }
61          catch (InterruptedException e) {
62              e.printStackTrace();
63          }
64      }
65  }
66
```

Fragment kodu 5: Plik `PCUtil.java` - klasa `PCUtil`

Eksperymenty przeprowadzono dla liczby producentów $m \in \{1, 5, 10, 50, 100, 500\}$, liczby konsumentów $n \in \{1, 5, 10, 50, 100, 500\}$ oraz parametru $M \in \{100, 500, 1000, 5000\}$. Wątki `Producer` oraz `Consumer` działają w zapętleniu, dlatego czas pomiaru statystyk w każdym przypadku był ograniczony do 5000ms. Eksperymenty przeprowadzono pięciokrotnie w celu uśrednienia wyników.

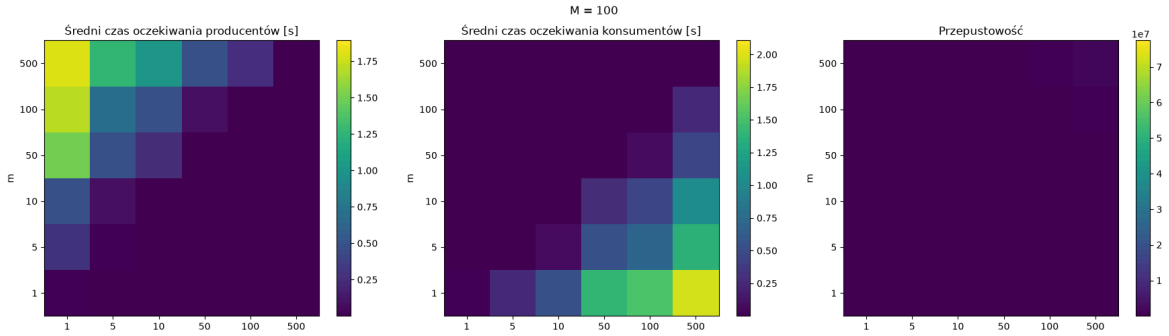
W trakcie doświadczenia zmierzono trzy statystyki:

- **Średni czas oczekiwania producentów** (`avg_producer_wait_time`) - średni czas, jaki wątki `Producer` spędziły na oczekiwaniu na dostęp do bufora [s].
- **Średni czas oczekiwania konsumentów** (`avg_consumer_wait_time`) - średni czas, jaki wątki `Consumer` spędziły na oczekiwaniu na dostęp do bufora [s].
- **Przepustowość** (`throughput`) - liczba elementów wstawionych oraz pobranych z bufora w czasie pomiaru.

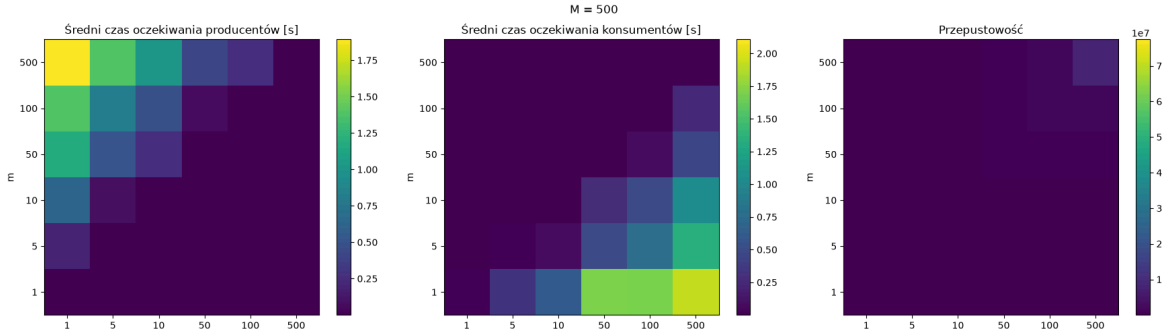
Każdą ze statystyk wyznaczono dla wszystkich kombinacji parametrów m, n oraz M.

4. Wyniki

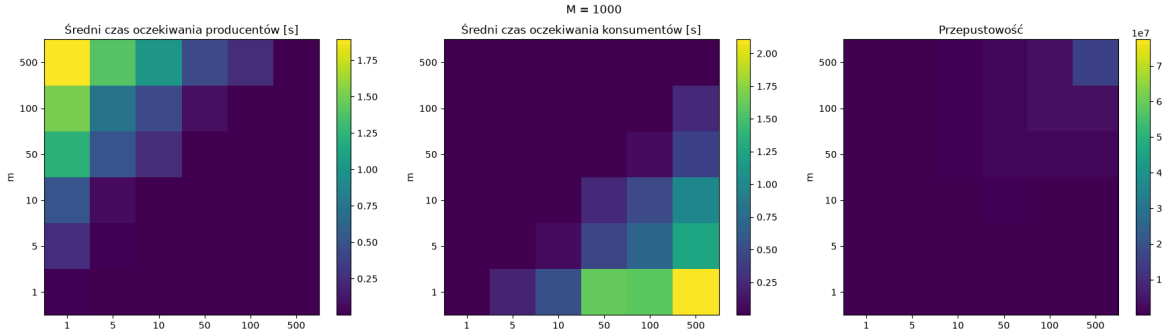
Na rysunkach nr 1-4 przedstawiono statystyki eksperymentów dla różnych wartości parametru M. Wykresy tych samych statystyk przedstawiono w tej samej skali.



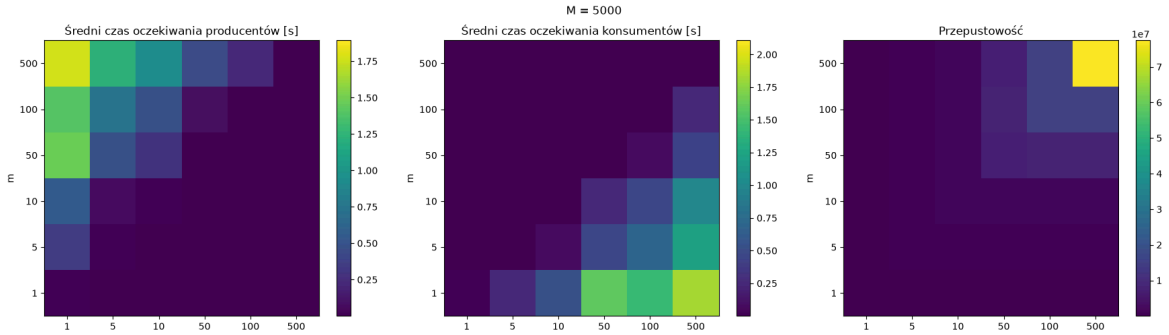
Rysunek 1: Mapy cieplne statystyk dla parametru $M = 100$



Rysunek 2: Mapy cieplne statystyk dla parametru $M = 500$



Rysunek 3: Mapy cieplne statystyk dla parametru $M = 1000$



Rysunek 4: Mapy cieplne statystyk dla parametru $M = 5000$

5. Wnioski

Średni czas oczekiwania producentów oraz konsumentów na dostęp do bufora nie zależy od parametru M (wielkości bufora $2M$).

Zależy on natomiast od stosunku liczby producentów do liczby konsumentów. Gdy $m > n$, oczekują producenci (bufor jest najczęściej pełny). Gdy $n > m$, oczekują konsumenci (bufor jest najczęściej pusty). Dla $m \approx n$ czasy oczekiwania obu stron są bliskie zeru.

Przepustowość systemu rośnie w przybliżeniu liniowo wraz z $\min(m, n)$ oraz wraz z parametrem M, ograniczającym rozmiar bufora oraz liczbę elementów, które producent/konsument może przetworzyć w jednej operacji.

Bibliografia

[1] Oracle, „Concurrency Utilities Overview”. Dostęp: 30 października 2026. [Online]. Dostępne na: https://docs.oracle.com/javase/1.5.0/docs/guide/concurrency/overview.html